Интеграция GitLab с Kubernetes

Занятие 11 · День 5, после обеда

`git push` → тесты → образ → деплой в кластер

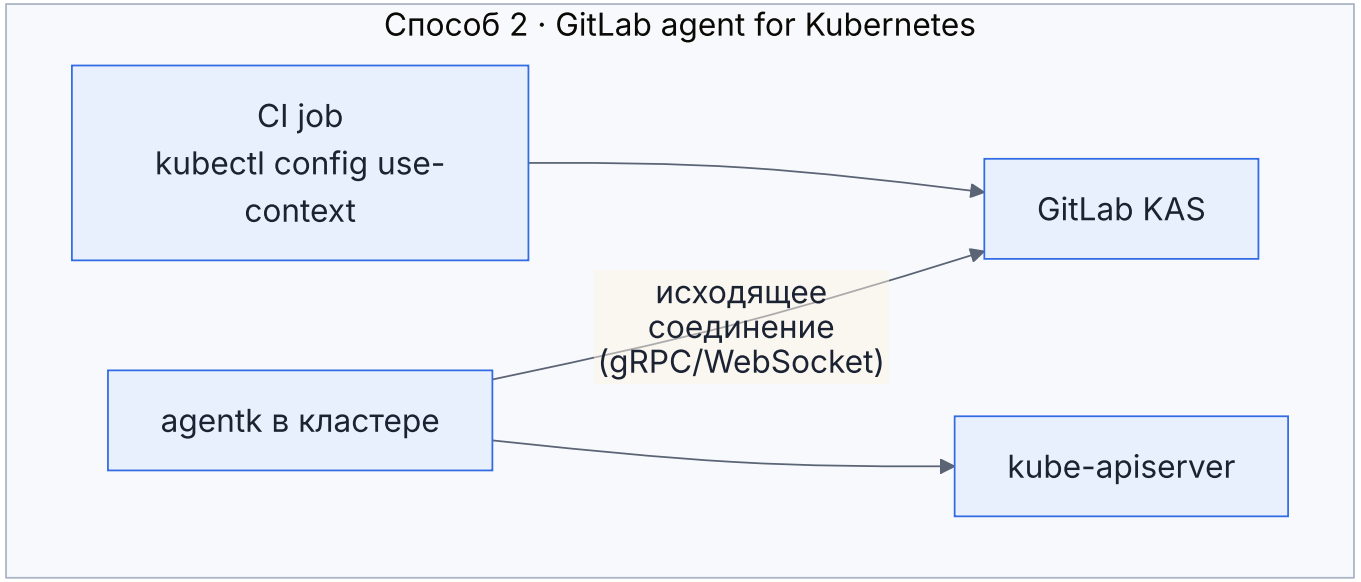
Что нужно связать

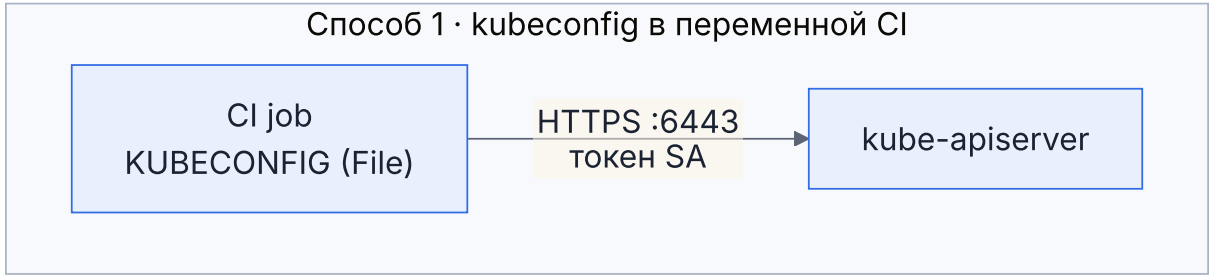
1. **Кластер → Registry**: узлы должны скачать приватный образ
2. **CI → Кластер**: job должен иметь права на деплой — **и только их**
3. **Окружения**: staging автоматически, production — по решению человека
4. **Обратная связь**: GitLab знает, какая версия где развёрнута

Три способа дать CI доступ к кластеру

Способ	Как	Когда
kubeconfig в CI-переменной	ServiceAccount + RBAC + токен, переменная типа File	просто, понятно, API кластера доступен runner'y
GitLab agent for Kubernetes	`agentk` в кластере сам подключается к GitLab (KAS)	кластер за NAT/firewall, много кластеров/проектов
Runner внутри кластера	executor kubernetes + ServiceAccount runner'a	платформа на k8s

Способы 1 и 2 на схеме





Способ 1

kubeconfig сервисного аккаунта

Принцип наименьших привилегий

Плохо: положить `admin.conf` в переменную CI — любой, кто может править `.gitlab-ci.yml`, получает **cluster-admin**.

Хорошо:

- Отдельный **ServiceAccount** `gitlab-deployer`
- Права только в нужных namespace: `shop-staging`, `shop-prod`
- Встроенная ClusterRole `edit` через **RoleBinding** (не ClusterRoleBinding!)
- Namespace создаёт **администратор**, а не CI
- Переменная **protected** — недоступна в feature-ветках

RBAC для CI

```yaml
apiVersion: v1
kind: ServiceAccount
metadata: { name: gitlab-deployer, namespace: gitlab-ci }
---
apiVersion: v1
kind: Secret                                  # долгоживущий токен SA
metadata:
  name: gitlab-deployer-token
  namespace: gitlab-ci
  annotations: { kubernetes.io/service-account.name: gitlab-deployer }
type: kubernetes.io/service-account-token
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: gitlab-deployer-edit, namespace: shop-staging }
subjects:
  - { kind: ServiceAccount, name: gitlab-deployer, namespace: gitlab-ci }
roleRef: { apiGroup: rbac.authorization.k8s.io, kind: ClusterRole, name: edit }
# + такой же RoleBinding в shop-prod
```

```
kubectl auth can-i create deployments -n shop-staging --as=system:serviceaccount:gitlab-ci:gitlab-deployer  # yes
kubectl auth can-i get secrets -n kube-system       --as=system:serviceaccount:gitlab-ci:gitlab-deployer  # no
```

ДЕМО kubeconfig для CI

```
kubectl apply -f gitlab-deployer.yaml
./make-kubeconfig.sh > gitlab-deployer.kubeconfig

KUBECONFIG=gitlab-deployer.kubeconfig kubectl get pods -n shop-staging   # ok
KUBECONFIG=gitlab-deployer.kubeconfig kubectl get nodes                  # Forbidden ✓
```

GitLab → проект → **Settings → CI/CD → Variables → Add variable**:

Поле	Значение
Key	`KUBECONFIG`
Type	**File**
Value	содержимое `gitlab-deployer.kubeconfig`
Flags	**Protected**, Masked нельзя (многострочное) — поэтому File + Protected

`kubectl` и `helm` сами читают переменную окружения `KUBECONFIG` → путь к файлу.

Кластер и приватный registry

Узлу нужно **скачать** образ из приватного проекта:

1. containerd знает, что registry — HTTP → настроили в Kubespray (`hosts.toml`) ✓
2. Нужны **учётные данные** → **Deploy token** проекта:
   Settings → Repository → Deploy tokens → имя **`gitlab-deploy-token`**, scope `read_registry`
   → GitLab сам передаёт его в jobs как `CI_DEPLOY_USER` / `CI_DEPLOY_PASSWORD`
3. CI создаёт Secret типа `dockerconfigjson` в namespace и указывает его в поде:

```
kubectl -n "$NAMESPACE" create secret docker-registry gitlab-registry \
  --docker-server="$CI_REGISTRY" \
  --docker-username="$CI_DEPLOY_USER" --docker-password="$CI_DEPLOY_PASSWORD" \
  --dry-run=client -o yaml | kubectl apply -f -
```

```
imagePullSecrets: [{ name: gitlab-registry }]
```

Деплой-jobs

```yaml
.deploy:
  stage: deploy
  image: alpine/k8s:1.36.4                # kubectl + helm
  resource_group: $CI_ENVIRONMENT_NAME
  script:
    - kubectl -n "$NAMESPACE" create secret docker-registry gitlab-registry
      --docker-server="$CI_REGISTRY" --docker-username="$CI_DEPLOY_USER"
      --docker-password="$CI_DEPLOY_PASSWORD" --dry-run=client -o yaml | kubectl apply -f -
    - helm upgrade --install shop-api chart/ --namespace "$NAMESPACE"
      --set image.repository="$CI_REGISTRY_IMAGE" --set image.tag="$CI_COMMIT_SHORT_SHA"
      --set "imagePullSecrets[0].name=gitlab-registry" --wait --timeout 3m

deploy-staging:
  extends: .deploy
  variables: { NAMESPACE: shop-staging }
  environment: { name: staging }
  rules: [{ if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH }]

deploy-production:
  extends: .deploy
  variables: { NAMESPACE: shop-prod }
  environment: { name: production }
  rules: [{ if: $CI_COMMIT_TAG, when: manual }]
```

Итоговый пайплайн shop-api

```
push в feature-ветку / MR  →  test: unit-tests · lint-dockerfile · lint-chart
merge в main               →  test → build-image → deploy-staging (авто)
тег v1.2.0                 →  test → build-image → deploy-production (► вручную)
```

- Production-деплой видят **Operate → Environments**: какая версия, когда, кем
- **Rollback** в Environments → «Re-deploy» предыдущего успешного деплоя (или `helm rollback` вручную)
- Тег `v*` — **protected**, переменная `KUBECONFIG` — **protected** → в production может выкатить только Maintainer

Способ 2

GitLab agent for Kubernetes

GitLab agent for Kubernetes

- **agentk** — под в вашем кластере, сам открывает **исходящее** соединение к **KAS** (GitLab)
- Кластеру **не нужен** входящий доступ из GitLab/runner'а — работает за NAT и firewall
- Нет долгоживущих kubeconfig в переменных CI
- Возможности:
  - **CI/CD workflow** — jobs получают kube-контекст агента
  - **GitOps** с **Flux** — кластер сам тянет изменения из Git
  - Дашборд Kubernetes прямо в GitLab (Environments)

ДЕМО Регистрация агента

1. В репозитории — файл конфигурации агента:

```
# .gitlab/agents/shop-agent/config.yaml
ci_access:
  projects:
    - id: demo/shop-api        # каким проектам можно использовать агента в CI
```

2. **Operate → Kubernetes clusters → Connect a cluster** → агент `shop-agent` → GitLab покажет **готовую команду helm** с токеном `glagent-…`:

```
helm repo add gitlab https://charts.gitlab.io
helm upgrade --install shop-agent gitlab/gitlab-agent \
  --namespace gitlab-agent-shop-agent --create-namespace \
  --set config.token=glagent-XXXX \
  --set config.kasAddress=ws://kas.192.168.56.200.nip.io   # адрес — как покажет UI
```

3. Статус агента в UI — **Connected**

Деплой через агента

```yaml
deploy-staging-agent:
  stage: deploy
  image: alpine/k8s:1.36.4
  script:
    - kubectl config get-contexts
    - kubectl config use-context demo/shop-api:shop-agent   # <путь проекта>:<агент>
    - helm upgrade --install shop-api chart/ -n shop-staging --set image.tag=$CI_COMMIT_SHORT_SHA --wait
  environment:
    name: staging
```

- GitLab сам подкладывает в job kubeconfig с контекстом агента
- Права по умолчанию — права ServiceAccount агента (чарт даёт **широкие**!) → ограничивайте:
  - `ci_access … access_as: { ci_job: {} }` — **impersonation**, права выдаются RBAC-ом на группы `gitlab:ci_job`, `gitlab:project_env:…`

Push vs Pull (GitOps)

	Push (CI деплоит)	Pull (GitOps)
Кто применяет	job в пайплайне	оператор в кластере (Flux, Argo CD)
Доступ к кластеру	у CI	только у оператора
Дрейф (ручные правки)	не замечается	автоматически исправляется
Источник правды	пайплайн + Git	**только Git**
Сложность	ниже	выше

GitOps-цепочка: CI собирает образ → **коммитит новый тег** в репозиторий с манифестами → Flux/Argo CD применяет.

Безопасность интеграции

- Отдельный ServiceAccount на проект/окружение, `edit` в своих namespace — не `cluster-admin`
- **Protected** переменные и теги для production; **protected runner**
- Deploy token только `read_registry`; срок жизни токенов
- Не печатать секреты (`set -x`, `cat $KUBECONFIG` в логах — запрещено)
- Политики в кластере: ResourceQuota, NetworkPolicy, Pod Security Standards
- Сканирование образов и манифестов в пайплайне (Trivy, Kubescape, SAST)

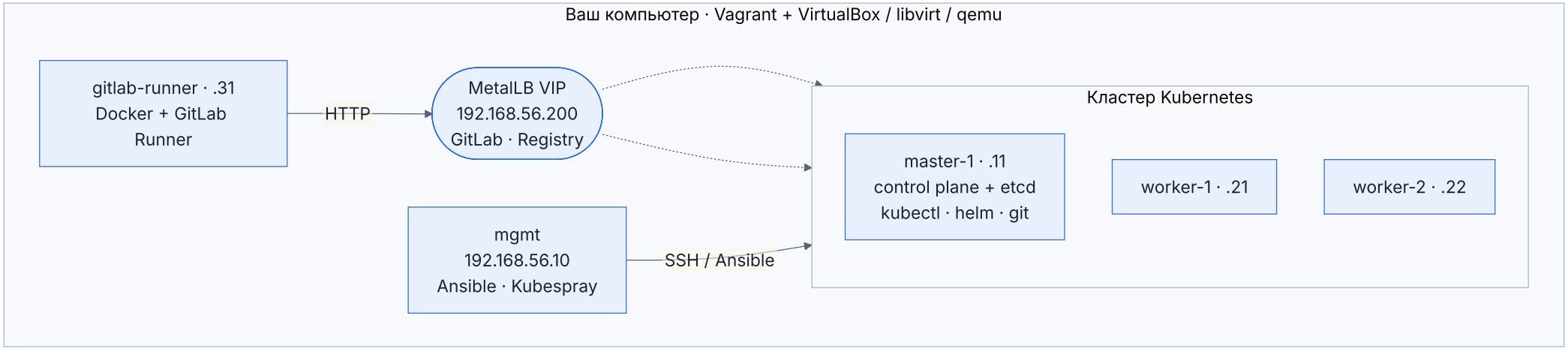
ДЕМО Демо: от коммита до production

```
git switch -c feature/greeting
sed -i 's/Добро пожаловать в магазин!/Добро пожаловать в shop v2!/' chart/values.yaml
git commit -am "feat: новое приветствие" && git push -u origin feature/greeting
#   → MR: пайплайн test (без деплоя) → review → merge

#   → main: build-image → deploy-staging
kubectl -n shop-staging get pods,svc
curl http://<EXTERNAL-IP-staging>/

git switch main && git pull
git tag -a v1.1.0 -m "Релиз 1.1.0" && git push origin v1.1.0
#   → тег: build-image → ► deploy-production (вручную в UI)
helm -n shop-prod history shop-api
```

Курс: что мы построили



Kubespray-кластер · MetalLB · GitLab в Kubernetes (Gateway API) · Runner на BM · Container Registry · Helm-чарт · пайплайн test → build → deploy (staging / production)

Куда двигаться дальше

- **Наблюдаемость**: Prometheus, Grafana, Loki, OpenTelemetry
- **GitOps**: Argo CD или Flux
- **Секреты**: Vault, External Secrets Operator, Sealed Secrets
- **Безопасность**: Pod Security Standards, Kyverno / OPA Gatekeeper, Trivy
- **Сеть**: NetworkPolicy, Cilium, service mesh
- **TLS**: cert-manager + Let's Encrypt / внутренний CA
- Сертификации: **CKA**, **CKAD**, **CKS**; GitLab Certified CI/CD Associate

Итоги курса

- Kubernetes: архитектура, установка **Kubespray**, сущности, **манифесты**, первое приложение
- **Git**: ветки, MR, теги
- **GitLab**: в Kubernetes, проекты, **runner**, `.gitlab-ci.yml`, **Helm**
- **Интеграция**: ServiceAccount + RBAC + kubeconfig / GitLab agent, приватный registry, окружения

> Вы прошли весь путь от `vagrant up` до автоматического деплоя по `git push`. Спасибо!

Практика 11

Автоматический деплой notes-api в кластер

Задание — на платформе testlms.kz